

0. 先前准备

由于 rv1106 是 armv7l 架构的，并不是 arm64 的，需要找 arm 的交叉编译链。
而且移植的 buildroot 缺少一些库，后面也要复制到板卡才行。

0.1 编译链安装

在 x86_64 的 pc 主机上进行下面的步骤，我使用的是 ubuntu20.04:

这个是鲁班猫的 arm 交叉编译链的仓库，先拉取下来:

https://github.com/LubanCat/gcc-arm-10.3-2021.07-x86_64-arm-none-linux-gnueabi

拉取下来之后解压，我解压到了 /opt 目录下。打开 ~/.bashrc，配置环境变量，如图所示:



```
$ .bashrc
25  else
30      fi
31  fi
32  unset __conda_setup
33  # <<< conda initialize <<<
34
35
36  export PATH=/home/dev/356x_sdk/prebuilts/gcc/linux-x86/aarch64/gcc-linaro-6.3.1-
37
38  export ARCH=arm
39  export CROSS_COMPILE=arm-none-linux-gnueabi-
40  export PATH=$PATH:/opt/gcc-arm-10.3-2021.07-x86_64-arm-none-linux-gnueabi/bin
```

```
export ARCH=arm
```

```
export CROSS_COMPILE=arm-none-linux-gnueabi-
```

```
export PATH=$PATH:/opt/gcc-arm-10.3-2021.07-x86_64-arm-none-linux-gnueabi/bin
```

上面三行粘贴到 .bashrc 文件末尾，保存后执行 `source ~/.bashrc` 重新加载配置

加载配置后执行 `arm-none-linux-gnueabi-gcc -v` 查看版本，有版本输出就说明配置成功了

0.2 安装所需的库文件

ffmpeg 和 x264 需要 libc6-armhf-cross 这个库，但是直接在 ubuntu20.04 获取到的版本不够高，所以需要添加高版本的源:

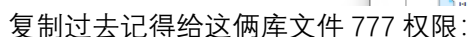
执行 `sudo vim /etc/apt/sources.list`，在里面添加高版本的源:

...

```
deb http://mirrors.aliyun.com/ubuntu/ jammy main    #添加该行到文件
```

...

下载成功后/usr 目录下，会有 arm-linux-gnueabihf 文件夹，里面有个 lib 文件夹，用 zip 或者别的什么压缩出来，这里压缩起来只是方便保存，不做也行，反正后面只是复制粘贴到对应的目录而已。把 lib 文件夹里面的所有文件全部粘贴到板卡的/lib 文件夹里，如图：



```
# cd /lib
# ls -l
total 3824
drwxr-xr-x  3 1000    1000          4096 Jun  4 06:35 dhcpcd
-rwxrwxrwx  1 root    root          134740 Aug 14 03:44 ld-linux-armhf.so.3
-rwxrwxrwx  1 root    root          134740 Aug 14 03:44 ld-linux.so.3
-rwxr-xr-x  1 1000    1000          32608 Jun  4 06:30 ld-ucLibc-1.0.31.so
lrwxrwxrwx  1 1000    1000           14 Jun  4 06:24 ld-ucLibc.so.0 -> ld-ucLibc.so.1
lrwxrwxrwx  1 1000    1000           19 Jun  4 06:24 ld-ucLibc.so.1 -> ld-ucLibc-1.0.31.s
o
-rw-----  1 root    root          5580 Aug 14 03:50 libBrokenLocale.so.1
-rw-----  1 root    root          9812 Aug 14 03:50 libanl.so.1
lrwxrwxrwx  1 1000    1000           18 Apr  7 2022 libatomic.so -> libatomic.so.1.2.0
lrwxrwxrwx  1 1000    1000           18 Jun  4 06:24 libatomic.so.1 -> libatomic.so.1.2.0
```

<https://wiki.lckfb.com/zh-hans/tspi-rk3566/documentation/buildroot-add-ffmpeg.html#%E3%80%90%E7%AB%8B%E5%88%9B%C2%B7%E6%B3%B0%E5%B1%B1%E6%B4%B%Erk3566%E5%BC%80%E5%8F%91%E6%9D%BF%E3%80%91buildroot%E6%B7%BB%E5%8A%A0ffmpeg>

先说 x264 的, 主要是交叉编译链的位置和前缀, 以及 ./configure 的 --host 配置需要改一下, 安装位置可以和教程一样。如下图:

```
$ autoConfig_armhf.sh ~/x264 X $ autoConfig.sh $ autoConfig_armhf.sh ~/ffmpeg-rockchip
buildroot > x264 > $ autoConfig_armhf.sh
1  #!/bin/bash
2
3  # 定义变量
4  # 设置x264的安装位置
5  PREFIX="./x264build_armhf"
6
7  # 交叉编译器bin路径,使用aarch64-linux-gnu-gcc -v命令查看
8  TOOLCHAIN_DIR="/opt/gcc-arm-10.3-2021.07-x86_64-arm-none-linux-gnueabi/bin"
9
10 # 工具链前缀,用于指定交叉编译工具的路径
11 TOOLCHAIN="$TOOLCHAIN_DIR/arm-none-linux-gnueabi-"
12
13 # 运行 configure 脚本
14 echo "正在配置 x264..."
15 ./configure \
16     --prefix=$PREFIX \
17     --disable-asm \
18     --enable-shared \
19     --enable-static \
20     --host=arm-linux \
21     --cross-prefix=$TOOLCHAIN \
22     --enable-pic
23
24 # 检查 configure 的返回状态
```

接下来讲一下 ffmpeg 的区别，主要是./configure 的--cc 和--arch 的区别，如下图：

```
buildroot > ffmpeg-rockchip > $ autoConfig_armhf.sh
12 # 设置 PKG_CONFIG_PATH 环境变量
13 export PKG_CONFIG_PATH=$X264_LIB_DIR/pkgconfig:$PKG_CONFIG_PATH
14
15 echo "配置环境变量："
16 echo "export PKG_CONFIG_PATH=$X264_LIB_DIR/pkgconfig:$PKG_CONFIG_PATH"
17
18 echo "正在配置 ./configure..."
19
20 ./configure \
21     --prefix=$PREFIX \
22     --enable-shared \
23     --disable-static \
24     --enable-cross-compile \
25     --arch=arm \
26     --disable-stripping \
27     --target-os=linux \
28     --cc=arm-none-linux-gnueabi-gcc \
29     --enable-libx264 \
30     --extra-cflags=-I$X264_INCLUDE_DIR \
31     --extra-ldflags=-L$X264_LIB_DIR \
32     --pkg-config-flags="--static" \
33     --extra-libs="-lpthread -lm" \
34     --enable-gpl \
35     --enable-pic \
36     --enable-nonfree \
37     --enable-pthreads
38
```

修改完按上面的参考文档正常编译，复制 bin 和 lib 的文件到板卡。

最终效果如下图：

```

Password:
# ffmpeg -version
ffmpeg version 6.1 Copyright (c) 2000-2023 the FFmpeg developers
built with gcc 10.3.1 (GNU Toolchain for the A-profile Architecture 10.3-2021.07 (arm-10.29))
configuration: --prefix=../ffmpeg_rk_build_armhf --enable-shared --disable-static --enable-cr
ss-compile --arch=armv7l --disable-stripping --target-os=linux --cc=arm-none-linux-gnueabi
gcc --enable-libx264 --extra-cflags=-I/home/dev/buildroot/x264/x264build_armhf/include --ext
a-ldflags=-L/home/dev/buildroot/x264/x264build_armhf/lib --pkg-config-flags=--static --extra
libs='-lpthread -lm' --enable-gpl --enable-pic --enable-nonfree --enable-pthreads
libavutil      58. 29.100 / 58. 29.100
libavcodec     60. 31.102 / 60. 31.102
libavformat    60. 16.100 / 60. 16.100
libavdevice    60.  3.100 / 60.  3.100
libavfilter     9. 12.100 /  9. 12.100
libswscale     7.  5.100 /  7.  5.100
libswresample  4. 12.100 /  4. 12.100
libpostproc   57.  3.100 / 57.  3.100
# x264 --help
x264 core:165
Syntax: x264 [options] -o outfile infile

Infile can be raw (in which case resolution is required),
    or YUV4MPEG (*.y4m),
    or Avisynth if compiled with support (yes).
    or libav* formats if compiled with lavf support (no) or ffms support (no).
Outfile type is selected by filename:
.264 -> Raw bytestream
.mkv  -> Matroska
.flv  -> Flash Video
.mp4  -> MP4 if compiled with GPAC or L-SMASH support (no)
Output bit depth: 8/10

Options:

    -h, --help                List basic options
    --longhelp                List more options
    --fullhelp                List all options

Example usage:

    Constant quality mode:
        x264 --crf 24 -o <output> <input>
```